4 Managing passwords

This chapter covers

- Implementing and working with the `PasswordEncoder`
- Using the tools offered by the Spring Security Crypto module

In chapter 3, we discussed managing users in an application implemented with Spring Security. But what about passwords? They're certainly an essential piece of the authentication flow. In this chapter, you'll learn how to manage passwords and secrets in an application implemented with Spring Security. We'll discuss the `PasswordEncoder` contract and the tools offered by the Spring Security Crypto module (SSCM) for the management of passwords.

4.1 Using password encoders

From chapter 3, you should now have a clear image of what the `UserDetails` interface is, as well as multiple ways to use its implementation. But as you learned in chapter 2, different actors manage user representation during the authentication and authorization processes. You also learned that some of these have defaults, like `UserDetailsService` and `PasswordEncoder`. You now know that you can override the defaults. We continue with a deep understanding of these beans and ways to implement them, so in this section, we will analyze the `PasswordEncoder`. Figure 4.1 reminds you of where the `PasswordEncoder` fits into the authentication process.

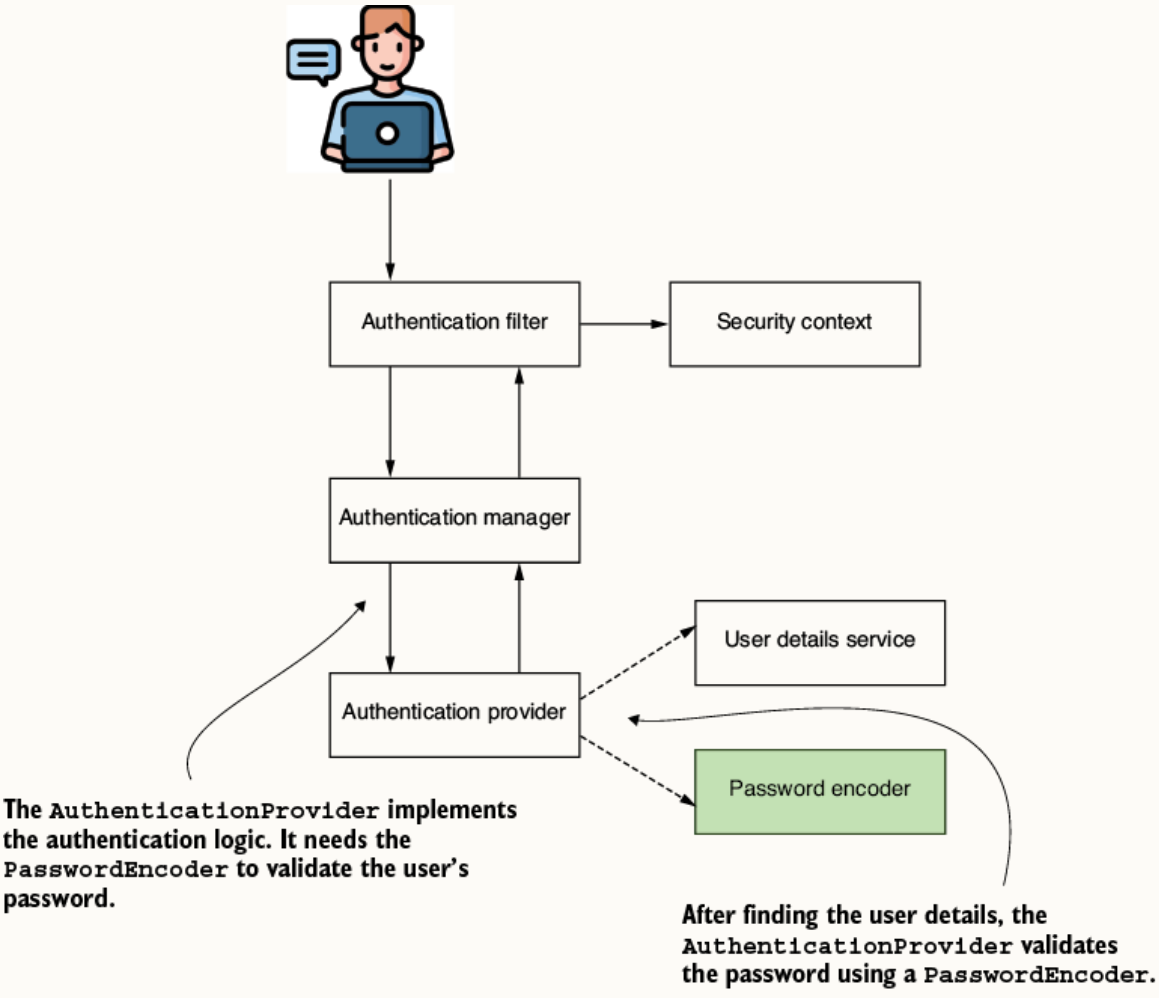


Figure 4.1 The Spring Security authentication process. The `AuthenticationProvider` uses the `PasswordEncoder` to validate the user's password in the authentication process.

Because in general, a system doesn't manage passwords in plain text, these usually undergo a sort of transformation that makes it more challenging to read and steal them. For this responsibility, Spring Security defines a separate contract. To explain it simply in this section, I provide plenty of code examples related to the `PasswordEncoder` implementation. We'll start with understanding the contract, and then we'll write our implementation in a project. Then, in section 4.1.3, I'll provide you with a list of the most well-known and widely used implementations of `PasswordEncoder` provided by Spring Security.

4.1.1 The PasswordEncoder contract

In this section, we discuss the definition of the `PasswordEncoder` contract. You implement this contract to tell Spring Security how to validate a user's password. In the authentication process, the `PasswordEncoder` decides whether a password is valid. Every system stores passwords encoded in some way. You preferably store them hashed so that there's no chance someone can read them. The `PasswordEncoder` can also encode passwords. The methods `encode()` and `matches()`, which the contract declares, are actually the definition of its responsibility. Both are parts of the same contract because these are strongly interlinked. The way the application encodes a password is related to the way the password is validated. Let's first review the contents of the `PasswordEncoder` interface:

```
public interface PasswordEncoder {

  String encode(CharSequence rawPassword);
  boolean matches(CharSequence rawPassword, String encodedPassword);

  default boolean upgradeEncoding(String encodedPassword) {
    return false;
  }
}
```

The interface defines two abstract methods and one with a default implementation. The abstract `encode()` and `matches()` methods are also the ones that you most often hear about when dealing with a `PasswordEncoder` implementation.

The purpose of the `encode(CharSequence rawPassword)` method is to return a transformation of a provided string. In terms of Spring Security functionality, it's used to provide encryption or a hash for a given password. You can use the `matches(CharSequence rawPassword, String encodedPassword)` method afterward to check whether an encoded string matches a raw password. You use the `matches()` method in the authentication process to test a provided password against a set of known credentials. The third method, called `upgradeEncoding(CharSequence encodedPassword)`, defaults to false in the contract. If you override it to return true, then the encoded password is encoded again for better security.

In some cases, encoding the encoded password can make it more challenging to obtain the cleartext password from the result. In general, this is a kind of obscurity that I personally don't like. But the framework offers this possibility if you think it applies to your case.

4.1.2 Implementing your PasswordEncoder

As you observed, the two methods `matches()` and `encode()` have a strong relationship. If you override them, they should always correspond in terms of functionality: a string returned by the `encode()` method should always be verifiable with the `matches()` method of the same `PasswordEncoder`. In this section, you'll implement the `PasswordEncoder` contract and define the two abstract methods declared by the interface. Knowing how to implement the `PasswordEncoder`, you can choose how the application manages passwords for the authentication process. The most straightforward implementation is a password encoder that considers passwords in plain text: that is, it doesn't do any encoding on the password.

Managing passwords in cleartext is what the instance of `NoOpPasswordEncoder` is precisely. We used this class in our first example in chapter 2. If you were to write your own, it would look something like the following listing.

Listing 4.1 The simplest implementation of a `PasswordEncoder`

```
public class PlainTextPasswordEncoder
  implements PasswordEncoder {

  @Override
  public String encode(CharSequence rawPassword) {
    return rawPassword.toString();                    ①
  }

  @Override
  public boolean matches(
    CharSequence rawPassword, String encodedPassword) {
      return rawPassword.equals(encodedPassword);     ②
  }
}
```

① We don't change the password; we just return it as is.

② Checks if the two strings are equal

The result of the encoding is always the same as the password. So to check if these match, you only need to compare the strings with `equals()`. A simple implementation of `PasswordEncoder` that uses the hashing algorithm SHA-512 looks like the next listing.

Listing 4.2 Implementing a `PasswordEncoder` that uses SHA-512

```
public class Sha512PasswordEncoder
  implements PasswordEncoder {

  @Override
  public String encode(CharSequence rawPassword) {
    return hashWithSHA512(rawPassword.toString());
  }

  @Override
  public boolean matches(
    CharSequence rawPassword, String encodedPassword) {
    String hashedPassword = encode(rawPassword);
    return encodedPassword.equals(hashedPassword);
  }

  // Omitted code

}
```

In listing 4.2, we use a method to hash the string value provided with SHA-512. I omit the implementation of this method in listing 4.2, but you can find it in listing 4.3. We call this method from the `encode()` method, which now returns the hash value for its input. To validate a hash against an input, the `matches()` method hashes the raw password in its input and compares it for equality with the hash against which it does the validation.

Listing 4.3 The implementation of the method to hash the input with SHA-512

```
private String hashWithSHA512(String input) {
 StringBuilder result = new StringBuilder();
 try {
  MessageDigest md = MessageDigest.getInstance("SHA-512");
  byte [] digested = md.digest(input.getBytes());
  for (int i = 0; i < digested.length; i++) {
   result.append(Integer.toHexString(0xFF & digested[i]));
  }
 } catch (NoSuchAlgorithmException e) {
  throw new RuntimeException("Bad algorithm");
 }
 return result.toString();
}
```

You'll learn better options to do this in the next section, so don't bother too much with this code for now.

4.1.3 Choosing from the provided PasswordEncoder implementations

While knowing how to implement your `PasswordEncoder` is powerful, you must also be aware that Spring Security already provides you with some advantageous implementations. If one of these matches your application, you don't need to rewrite it. In this section, we discuss the `PasswordEncoder` implementation options that Spring Security provides. These are

- `NoOpPasswordEncoder` —Doesn't encode the password but keeps it in cleartext. We use this implementation only for examples. Because it doesn't hash the password, *you should never use it in a real-world scenario*.
- `StandardPasswordEncoder` —Uses SHA-256 to hash the password. This implementation is now deprecated, and *you shouldn't use it for your new implementations*. It's deprecated because it uses a hashing algorithm that we don't consider strong enough anymore, but you might still find this implementation used in existing applications. Preferably, if you find it in existing apps, you should change it with some other, more powerful password encoder.
- `Pbkdf2PasswordEncoder` —Uses the password-based key derivation function 2 (PBKDF2).
- `BCryptPasswordEncoder` —Uses a bcrypt strong hashing function to encode the password.
- `SCryptPasswordEncoder` —Uses a scrypt hashing function to encode the password.

For more about hashing and these algorithms, you can find a good discussion in chapter 2 of *Real-World Cryptography* by David Wong (Manning, 2021) at **http://mng.bz/QRJw**.

Let's take a look at some examples of how to create instances of these types of `PasswordEncoder` implementations. The `NoOpPasswordEncoder` doesn't encode the password. It has an implementation similar to the `PlainTextPasswordEncoder` from our example in listing 4.1. For this reason, we only use this password encoder with theoretical examples. Also, the `NoOpPasswordEncoder` class is designed as a singleton. You can't call its constructor directly from outside the class, but you can use the `NoOpPasswordEncoder.getInstance()` method to obtain the instance of the class like this:

```
PasswordEncoder p = NoOpPasswordEncoder.getInstance();
```

The `StandardPasswordEncoder` implementation provided by Spring Security uses SHA-256 to hash the password. For the `StandardPasswordEncoder`, you can provide a secret used in the hashing process. You set the value of this secret by the constructor's parameter. If you choose to call the no-arguments constructor, the implementation uses the empty string as a value for the key. However, the `StandardPasswordEncoder` is deprecated now, and I don't recommend that you use it with your new implementations. You could find older applications or legacy code that still uses it, so you should be aware of it. The next code snippet shows how to create instances of this password encoder:

```
PasswordEncoder p = new StandardPasswordEncoder();
```

```
PasswordEncoder p = new StandardPasswordEncoder("secret");
```

Another option offered by Spring Security is the `Pbkdf2PasswordEncoder` implementation that uses the PBKDF2 for password encoding. To create instances of the `Pbkdf2PasswordEncoder`, you have the following option:

```
PasswordEncoder p =
  new Pbkdf2PasswordEncoder("secret", 16, 310000, Pbkdf2PasswordEncoder. SecretKeyFactoryAlgorithm.PBKDF2WithHmacSHA256);
```

The PBKDF2 is a pretty easy, slow-hashing function that performs an HMAC as many times as specified by an iterations argument. The first three parameters received by the last call are the value of a key used for the encoding process, the number of iterations used to encode the password, and the size of the hash. The second and third parameters can influence the strength of the result. The fourth parameter gives the hash width. You can choose the following options:

- PBKDF2WithHmacSHA1
- PBKDF2WithHmacSHA256
- PBKDF2WithHmacSHA512

It is possible to choose more or fewer iterations, as well as the length of the result. The longer the hash, the more powerful the password (the same is true for the hash width). However, be aware that performance is affected by these values: the more iterations, the more resources your application consumes. You should make a wise compromise between the resources consumed for generating the hash and the needed strength of the encoding.

NOTE In this book, I refer to several cryptography concepts that you might like to know more about. For relevant information on HMACs and other cryptography details, I recommend *Real-World Cryptography* by David Wong (Manning, 2021). Chapter 3 of that book provides detailed information about HMAC. You can find the book at **http://mng.bz/XqJG**.

Another excellent option offered by Spring Security is the `BCryptPasswordEncoder`, which uses a bcrypt strong hashing function to encode the password. You can instantiate the `BCryptPasswordEncoder` by calling the no-arguments constructor. However, you also have the option to specify a strength coefficient representing the log rounds (logarithmic rounds) used in the encoding process. Moreover, you can also alter the `SecureRandom` instance used for encoding:

```
PasswordEncoder p = new BCryptPasswordEncoder();
PasswordEncoder p = new BCryptPasswordEncoder(4);

SecureRandom s = SecureRandom.getInstanceStrong();
PasswordEncoder p = new BCryptPasswordEncoder(4, s);
```

The log rounds value that you provide affects the number of iterations the hashing operation uses. The number of iterations used is $2^{\log\ rounds}$. For the iteration number computation, the value for the log rounds can only be between 4 and 31. You can specify this by calling one of the second or third overloaded constructors, as shown in the previous code snippet.

The last option I present to you is the `SCryptPasswordEncoder` (figure 4.2). This password encoder uses a scrypt hashing function. For the `ScryptPasswordEncoder`, you have the option to create its instances as presented in figure 4.2.

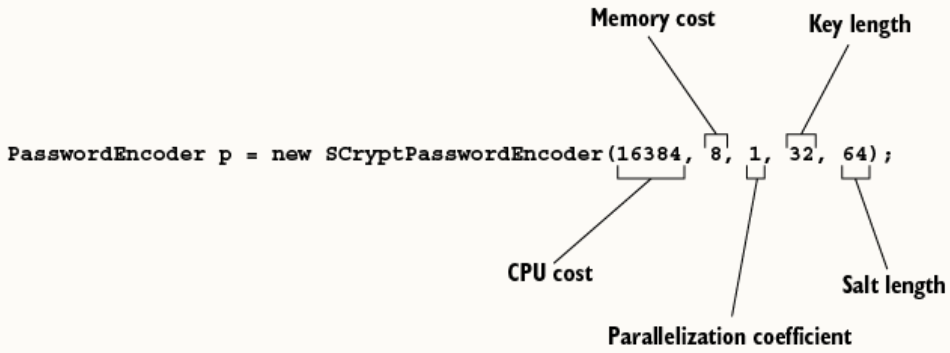


Figure 4.2 The `SCryptPasswordEncoder` constructor takes five parameters and allows you to configure CPU cost, memory cost, key length, and salt length.

4.1.4 Multiple encoding strategies with DelegatingPasswordEncoder

In this section, we discuss the cases in which an authentication flow must apply various implementations for matching the passwords. You'll also learn how to apply a useful tool that acts as a `PasswordEncoder` in your application. Instead of having its own implementation, this tool delegates to other objects that implement the `PasswordEncoder` interface.

In some applications, you might find it useful to have various password encoders and choose from these depending on some specific configuration. A common scenario in which I find the `DelegatingPasswordEncoder` in production applications is when the encoding algorithm is changed, starting with a particular version of the application. Imagine somebody finds a vulnerability in the currently used algorithm, and you want to change it for newly registered users, but you do not want to change it for existing credentials. You end up having multiple kinds of hashes. How do you manage this situation? While it isn't the only approach for this scenario, a good choice is to use a `DelegatingPasswordEncoder` object.

The `DelegatingPasswordEncoder` is an implementation of the `PasswordEncoder` interface that, instead of implementing its encoding algorithm, delegates to another instance of an implementation of the same contract. The hash starts with a prefix naming the algorithm used to define that hash. The `DelegatingPasswordEncoder` delegates to the correct implementation of the `PasswordEncoder` based on the prefix of the password.

It sounds complicated, but with an example, you can see that it is pretty easy. Figure 4.3 presents the relationship among the `PasswordEncoder` instances. The `DelegatingPasswordEncoder` has a list of `PasswordEncoder` implementations to which it delegates. The `DelegatingPasswordEncoder` stores each of the instances in a map. The `NoOpPasswordEncoder` is assigned to the key `noop`, while the `BCryptPasswordEncoder` implementation is assigned the key `bcrypt`. When the password has the prefix `{noop}`, the `DelegatingPasswordEncoder` delegates the operation to the `NoOpPasswordEncoder` implementation. If the prefix is `{bcrypt}`, then the action is delegated to the `BCryptPasswordEncoder` implementation, as presented in figure 4.4.

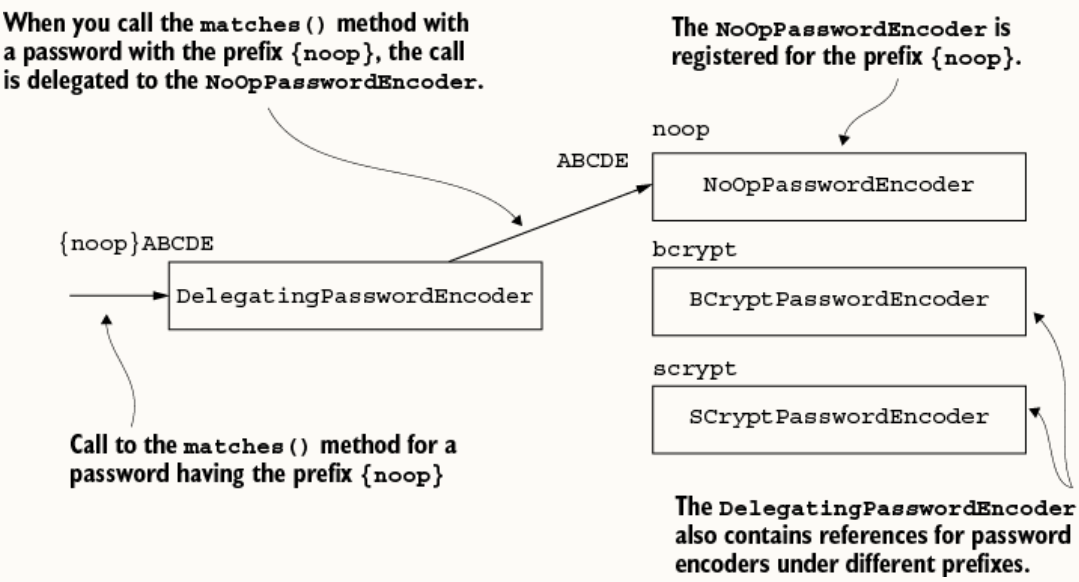


Figure 4.3 In this scenario, the `DelegatingPasswordEncoder` enlists a `NoOpPasswordEncoder` to handle passwords prefixed with `{noop}`, a `BCryptPasswordEncoder` for those beginning with `{bcrypt}`, and an `SCryptPasswordEncoder` for passwords starting with `{scrypt}`. When a password is accompanied by the `{noop}` prefix, the `DelegatingPasswordEncoder` directs the task to the `NoOpPasswordEncoder` version.

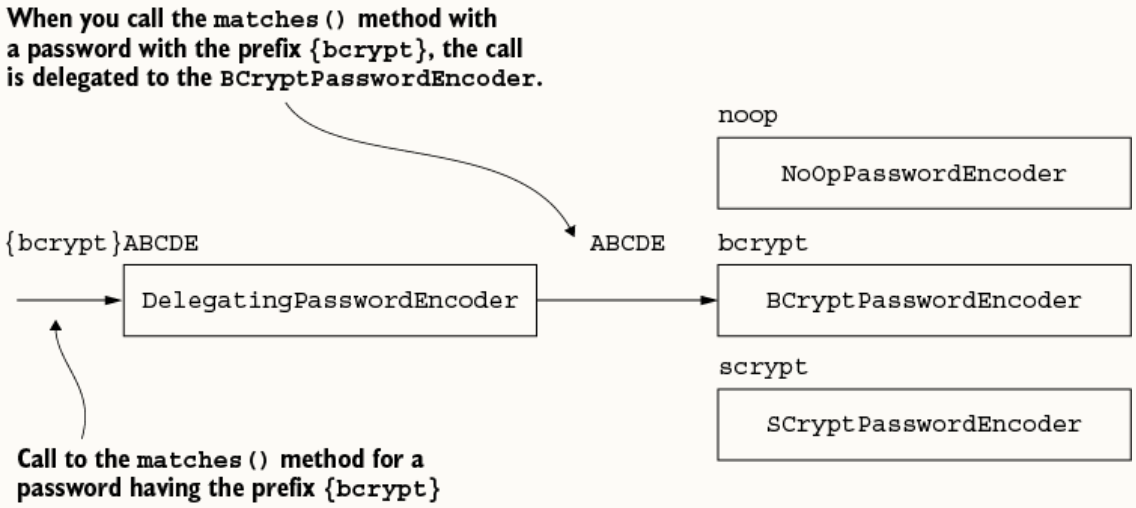


Figure 4.4 Here the `DelegatingPasswordEncoder` assigns the task of handling `{noop}`- prefixed passwords to the `NoOpPasswordEncoder`, `{bcrypt}`-prefixed passwords to the `BCryptPasswordEncoder`, and `{scrypt}`-prefixed passwords to the `SCryptPasswordEncoder`. If a password carries the `{bcrypt}` prefix, the `DelegatingPasswordEncoder` routes the process to the `BCryptPasswordEncoder`'s mechanism.

Next, let's find out how to define a `DelegatingPasswordEncoder`. You start by creating a collection of instances of your desired `PasswordEncoder` implementations, and you put these together in a `DelegatingPasswordEncoder`, as in the following listing.

Listing 4.4 Creating an instance of `DelegatingPasswordEncoder`

```
@Configuration
public class ProjectConfig {

 // Omitted code

 @Bean
 public PasswordEncoder passwordEncoder() {
  Map<String, PasswordEncoder> encoders = new HashMap<>();

  encoders.put("noop", NoOpPasswordEncoder.getInstance());
  encoders.put("bcrypt", new BCryptPasswordEncoder());
  encoders.put("scrypt", new SCryptPasswordEncoder());

  return new DelegatingPasswordEncoder("bcrypt", encoders);
```

```
    }
}
```

The `DelegatingPasswordEncoder` is just a tool that acts as a `PasswordEncoder`, so you can use it when you have to choose from a collection of implementations. In listing 4.4, the declared instance of `DelegatingPasswordEncoder` contains references to a `NoOpPasswordEncoder`, a `BCryptPasswordEncoder`, and an `SCryptPasswordEncoder`, and delegates the default to the `BCryptPasswordEncoder` implementation. Based on the prefix of the hash, the `DelegatingPasswordEncoder` uses the right `PasswordEncoder` implementation to match the password. This prefix has the key that identifies the password encoder to be used from the map of encoders. If there is no prefix, the `DelegatingPasswordEncoder` uses the default encoder. The default `PasswordEncoder` is the one given as the first parameter when constructing the `DelegatingPasswordEncoder` instance. For the code in listing 4.4, the default `PasswordEncoder` is bcrypt.

NOTE The curly braces are part of the hash prefix, and those should surround the name of the key. For example, if the provided hash is `{noop}12345`, the `DelegatingPasswordEncoder` delegates to the `NoOpPasswordEncoder` that we registered for the prefix `noop`. Again, remember that the curly braces are mandatory in the prefix.

If the hash looks like the next code snippet, the password encoder is the one we assign to the prefix `{bcrypt}`, which is the `BCryptPasswordEncoder`. This is also the one to which the application will delegate if there is no prefix at all because we defined it as the default implementation:

```
{bcrypt}$2a$10$xn3LI/AjqicFYZFruSwve.681477XaVNaUQbr1gioaWPn4t1KsnmG
```

For convenience, Spring Security offers a way to create a `DelegatingPasswordEncoder` that has a map to all the standard provided implementations of `PasswordEncoder`. The `PasswordEncoderFactories` class provides a `createDelegatingPasswordEncoder()` static method that returns an implementation of `DelegatingPasswordEncoder` with a full set of `PasswordEncoder` mappings and bcrypt as a default encoder:

```
PasswordEncoder passwordEncoder = PasswordEncoderFactories.createDelegatingPasswordEncoder();
```

Encoding vs. encrypting vs. hashing

In the previous sections, I often used the terms encoding, encrypting, and hashing. I want to briefly clarify these terms and the way they are used throughout the book.

Encoding refers to any transformation of a given input. For example, if we have a function `x` that reverses a string, function `x -> y` applied to `ABCD` produces `DCBA`.

Encryption is a particular type of encoding in which, to obtain the output, we provide both the input value and a key. The key makes it possible to choose afterward who should be able to reverse the function (obtain the input from the output). The simplest form of representing encryption as a function is

```
(x, k) -> y
```

where `x` is the input, `k` is the key, and `y` is the result of the encryption. This way, an individual who knows the key can use a known function to obtain the input from the output `(y, k) -> x`. We call this *reverse function decryption*. If the key used for encryption is the same as the one used for decryption, we usually call it a *symmetric* key.

If we have two different keys for encryption ($(x, k_1) \rightarrow y$) and decryption ($(y, k_2) \rightarrow x$), then we say that the encryption is done with *asymmetric* keys. Then (k_1, k_2) is called a *key pair*. The key used for encryption, k_1, is also referred to as the *public* key, while k_2 is known as the *private* key. This way, only the owner of the private key can decrypt the data.

Hashing is a particular type of encoding, except the function is only one way. That is, from an output `y` of the hashing function, you cannot get back the input `x`. However, there should always be a way to check if an output `y` corresponds to an input `x` so we can understand the hashing as a pair of functions for encoding and matching. If hashing is `x -> y`, then we should also have a matching function `(x,y) -> boolean`.

Sometimes, the hashing function could also use a random value added to the input: `(x, k) -> y`. We refer to this value as the *salt*. The salt makes the function stronger, enforcing the difficulty of applying a reverse function to obtain the input from the result.

To summarize the contracts we have discussed and applied up to now in this book, table 4.1 briefly describes each of the components.

Table 4.1 The interfaces that represent the main contracts for authentication flow in Spring Security

Contract	Description
`UserDetails`	Represents the user as seen by Spring Security.
GrantedAuthority	Defines an action within the purpose of the application that is allowable to the user (for example, read, write, delete, etc.).
`UserDetailsService`	Represents the object used to retrieve user details by username.
UserDetailsManager	A more particular contract for `UserDetailsService`. Besides retrieving the user by username, it can also be used to mutate a collection of users or a specific user.
`PasswordEncoder`	Specifies how the password is encrypted or hashed and how to check whether a given encoded string matches a plaintext password.

4.2 Taking advantage of the Spring Security Crypto module

In this section, we discuss the Spring Security Crypto module (SSCM), which is the part of Spring Security that deals with cryptography. Using encryption and decryption functions and generating keys isn't offered out of the box with the Java language, which constrains developers when adding dependencies that provide a more accessible approach to these features.

To make our lives easier, Spring Security also provides its own solution, which enables you to reduce the dependencies of your projects by eliminating the need to use a separate library. The password encoders are also part of the SSCM, even if we have treated them separately in previous sections. In this section, we discuss what other options the SSCM offers that are related to cryptography. You'll see examples of how to use two essential features from the SSCM:

- *Key generators*—Objects used to generate keys for hashing and encryption algorithms
- *Encryptors*—Objects used to encrypt and decrypt data

4.2.1 Using key generators

In this section, we discuss key generators. A *key generator* is an object used to generate a specific kind of key, generally required for an encryption or hashing algorithm. The implementations of key generators that Spring Security offers are great utility tools. You'll prefer to use these implementations rather than adding another dependency to your application, and this is why I recommend that you become familiar with them. Let's see some code examples of how to create and apply the key generators.

Two interfaces represent the two main types of key generators: `BytesKeyGenerator` and `StringKeyGenerator`. We can build them directly by making use of the factory class `KeyGenerators`. You can use a string key generator, represented by the `StringKeyGenerator` contract, to obtain a key as a string. Usually, we use this key as a salt value for a hashing or encryption algorithm. You can find the definition of the `StringKeyGenerator` contract in this code snippet:

```
public interface StringKeyGenerator {

   String generateKey();

}
```

The generator has only a `generateKey()` method that returns a string representing the key value. The next code snippet presents an example of how to obtain a `StringKeyGenerator` instance and how to use it to get a salt value:

```
StringKeyGenerator keyGenerator = KeyGenerators.string();
String salt = keyGenerator.generateKey();
```

The generator creates an 8-byte key, and it encodes that as a hexadecimal string. The method returns the result of these operations as a string. The second interface describing a key generator is the `BytesKeyGenerator`, which is defined as follows:

```
public interface BytesKeyGenerator {

 int getKeyLength();
 byte[] generateKey();

}
```

In addition to the `generateKey()` method that returns the key as a `byte[]`, the interface defines another method that returns the key length in number of bytes. A default `BytesKeyGenerator` generates keys of 8-byte length:

```
BytesKeyGenerator keyGenerator = KeyGenerators.secureRandom();
byte [] key = keyGenerator.generateKey();
int keyLength = keyGenerator.getKeyLength();
```

In the previous code snippet, the key generator generates keys of 8-byte length. If you want to specify a different key length, you can do this when obtaining the key generator instance by providing the desired value to the `KeyGenerators.secureRandom()` method:

```
BytesKeyGenerator keyGenerator = KeyGenerators.secureRandom(16);
```

The keys generated by the `BytesKeyGenerator` created with the `KeyGenerators.secureRandom()` method are unique for each call of the `generateKey()` method. In some cases, we prefer an implementation that returns the same key value for each call of the same key generator. In this case, we can create a `BytesKeyGenerator` with the `KeyGenerators.shared(int length)` method. In this code snippet, `key1` and `key2` have the same value:

```
BytesKeyGenerator keyGenerator = KeyGenerators.shared(16);
byte [] key1 = keyGenerator.generateKey();
byte [] key2 = keyGenerator.generateKey();
```

4.2.2 Encrypting and decrypting secrets using encryptors

In this section, we apply the implementations of encryptors that Spring Security offers with code examples. An *encryptor* is an object that implements an encryption algorithm. When talking about security, encryption and decryption are common operations, so expect to need these in your application.

We often need to encrypt data either when sending it between components of the system or when persisting it. The operations provided by an encryptor are encryption and decryption. There are two types of encryptors defined by the SSCM: `BytesEncryptor` and `TextEncryptor`. While they have similar responsibilities, they treat different data types. `TextEncryptor` manages data as a string. Its methods receive strings as inputs and return strings as outputs, as you can see from the definition of its interface:

```
public interface TextEncryptor {

  String encrypt(String text);
  String decrypt(String encryptedText);

}
```

The `BytesEncryptor` is more generic. You provide its input data as a byte array:

```
public interface BytesEncryptor {

  byte[] encrypt(byte[] byteArray);
  byte[] decrypt(byte[] encryptedByteArray);

}
```

Let's find out what options we have to build and use an encryptor. The factory class `Encryptors` offers us multiple possibilities. For `BytesEncryptor`, we could use the `Encryptors.standard()` or the `Encryptors.stronger()` methods like this:

```
String salt = KeyGenerators.string().generateKey();
String password = "secret";
String valueToEncrypt = "HELLO";

BytesEncryptor e = Encryptors.standard(password, salt);
byte [] encrypted = e.encrypt(valueToEncrypt.getBytes());
byte [] decrypted = e.decrypt(encrypted);
```

Behind the scenes, the standard byte encryptor uses 256-byte AES encryption to encrypt input. To build a stronger instance of the byte encryptor, you can call the `Encryptors.stronger()` method:

```
BytesEncryptor e = Encryptors.stronger(password, salt);
```

The difference is small and happens behind the scenes, where the AES encryption on 256-bit uses Galois/Counter Mode (GCM) as the mode of operation. The standard mode uses cipher block chaining (CBC), which is considered a weaker method.

`TextEncryptor`s come in three main types. You create these three types by calling the `Encryptors.text()` or `Encryptors.delux()`. Besides these methods to create encryptors, there is also a method that returns a dummy `TextEncryptor`, which doesn't encrypt the value. You can use the dummy `TextEncryptor` for demo examples or cases in which you want to test the performance of your application without spending time on encryption. The method that re-

turns this no-op encryptor is `Encryptors.noOpText()`. In the following code snippet, you'll find an example of using a `TextEncryptor`. Even if it is a call to an encryptor, in the example, `encrypted` and `valueToEncrypt` are the same:

```
String valueToEncrypt = "HELLO";
TextEncryptor e = Encryptors.noOpText();
String encrypted = e.encrypt(valueToEncrypt);
```

The `Encryptors.text()` encryptor uses the `Encryptors.standard()` method to manage the encryption operation, while the `Encryptors.delux()` method uses an `Encryptors.stronger()` instance like this:

```
String salt = KeyGenerators.string().generateKey();
String password = "secret";
String valueToEncrypt = "HELLO";

TextEncryptor e = Encryptors.text(password, salt);      ①
String encrypted = e.encrypt(valueToEncrypt);
String decrypted = e.decrypt(encrypted);
```

① Creates a TextEncryptor object that uses a salt and a password

Summary

- The `PasswordEncoder` has one of the most critical responsibilities in authentication logic—dealing with passwords.
- Spring Security offers several alternatives in terms of hashing algorithms, which makes the implementation only a matter of choice.
- Spring Security Crypto module (SSCM) offers various alternatives for the implementations of key generators and encryptors.
- Key generators are utility objects that help you generate keys used with cryptographic algorithms.
- Encryptors are utility objects that help you apply data encryption and decryption.

Previous chapter
< 3 Managing users

Next chapter
5 A web app's security begins with filters >